

DeepSeek爆火快来搭建私有ChatGPT

原创

码上来财

已于 2025-02-12 09:47:56 修改

阅读量2.2w

收藏 324

点赞数 136

CC 4.0 BY-SA版权

文章标签：

chatgpt

ai

docker

node.js

由于格式、图片和解析问题，问了更好的阅读体验，可前往 [阅读原文](#)

=====

最新的deepseek本地大模型部署，可查看 《[本地部署DeepSeek摆脱服务器繁忙](#)》一文

0723
个人部署，参考意义不大

DeepSeek最近火得连 OpenAI 都忍不住偷偷瞄了一眼，心想：“这哥们儿是从哪儿冒出来的？”要知道，OpenAI 可是 AI 界的“老大哥”，一直稳坐头把交椅，结果 DeepSeek 一登场，直接让硅谷的大佬们集体揉了揉眼睛，怀疑自己是不是看错了

美国社会各界对此的反应更是笑料百出。科技圈的大佬们一边喝着咖啡，一边在推特上疯狂刷屏：“DeepSeek 是什么鬼？为什么它的模型比我家的智能冰箱还聪明？”投资人们则忙得不可开交，一边疯狂给 DeepSeek 砸钱，一边暗自嘀咕：“这玩意儿会不会是下一个 OpenAI？我可不能再错过一次了！”甚至连普通网友都加入了讨论：“DeepSeek 是不是偷偷学了‘东方神秘力量’？怎么感觉它比 ChatGPT 还能聊？”

最后，连msk都忍不住发了一条推特：“DeepSeek？有意思……要不咱们也搞个 DeepSeek 5.0？”网友们纷纷调侃：“msk，你还是先把火箭发射成功再说吧！”

:::warning 小贴士
文章中涉及到的示例代码你都可以从 [这里查看](#)，若对你有帮助还望点赞支持
:::

AI热度

AI（人工智能）的热度近年来持续攀升，几乎成为科技、商业、学术和社会讨论的核心话题；AI也渐渐进入我们生活中的个个角落

🔔：

AI 配图



AI 生成



AI 生成



AI 生成



AI 生成

已为你生成图片，点击图片可查看大图并更换图片风格，点击“使用”可直接应用图片。

正在吃苹果的橘猫

2.35:1 

重新创作

已阅读并同意遵守《微信公众平台AI配图功能使用条款》及《微信公众平台个人信息保护指引》。

度娘AI搜索：



等等，还有很多很多产品或多或少都有AI的影子，可以说没有AI都觉得out了

AI 的前景充满机遇和挑战。它将成为推动社会进步的核心技术，改变我们的工作、生活和思维方式。然而，AI 的发展也需要在技术、伦理、法律和社会层面进行平衡，确保其造福全人类。未来，AI 不仅是工具，更是人类智慧的延伸，帮助我们解决更多复杂问题，创造更美好的世界

回归正题，DeepSeek都这么火了我们也尝试下用它搭建自己的ChatGPT吧~

为什么需要搭建私有AI

想象一下，AI 就像一位超级助手，能帮你处理各种任务，但如果你把所有的秘密都告诉一个“公共助手”，难免会担心隐私泄露。这就是为什么越来越多的企业和个人选择搭建私有 AI——它就像一位专属的私人助理，只为你服务，完全受你控制

私有 AI 的最大好处是数据安全。你可以把所有敏感信息（比如客户数据、商业机密）交给它，不用担心泄露给第三方。同时，私有 AI 还能根据你的需求量身定制，无论是行业特性还是具体任务，它都能完美适配。此外，私有 AI 运行在本地或私有云上，响应更快、更稳定，不会受网络波动或公有云资源限制的影响

前期准备

既然DeepSeek这么火那就是用它的模型来作为我们私有AI的核心数据源吧，因为DeepSeek是国内的所以对于国内用户来一系列的操作步骤都非常友好，赶紧上车！

注册

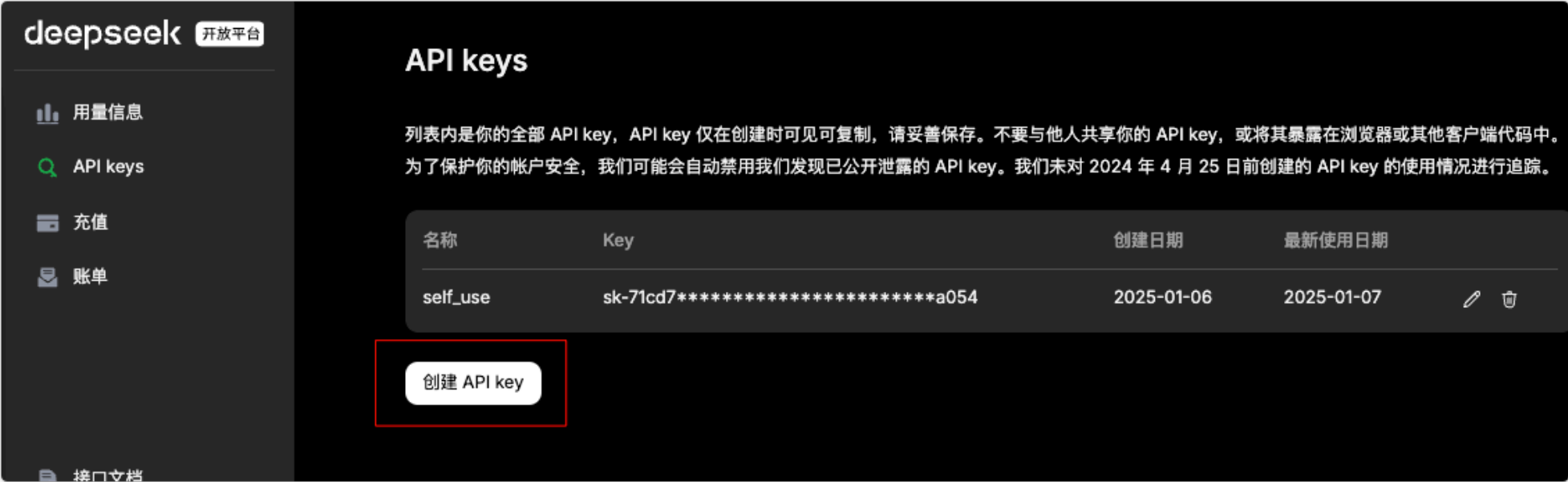
打开DeepSeek的官网，点击页面上的API接入，就会跳转到登录页面



新用户需要注册你可以使用微信扫码登录，但都需要绑定手机号

申请API KEY

调用大模型最关键的就是有对应的令牌，先创建一个令牌，名字最好有一定的寓意，避免长时间后忘记。创建后会生成对应的KEY，一定要将其复制保存到自己的资料库里，后面在控制台是无法查看的



充值

DeepSeek目前是给了500万token的免费使用（具体参考官网），免费使用有时效性大概1个月，但也足够我们玩了。后面如果你还想要使用的话，就需要充值了

充值对国内用户很友好也比较简单，但需要实名认证，简单填写姓名和ID Card就可以了

下面是我充值10元后，新增500万tokens



以上工作都准备好后就可以开始对接大模型了，这里我们使用NodeJS搭建应用

NodeJS搭建

调用大模型非常简单，官方给了一段简单的使用代码，可以查看 [DeepSeek API Docs](#)

这里我们使用NodeJS版本的代码，核心还是使用 [OpenAI Node SDK](#) 来作为基础载体，然后根据提供的模型地址来响应数据

安装openai依赖：

```
→ npm install openai
```

AI初体验

将官网的代码示例复制到对应的文件中，这里创建 `main.js`：

```
import OpenAI from "openai";
import { CHAT_CONFIG } from "../config.js";

const openai = new OpenAI({
  ...CHAT_CONFIG,
  baseUrl: 'https://api.deepseek.com',
```

```
// apiKey: '<DeepSeek API Key>'
});

async function main() {
  const completion = await openai.chat.completions.create({
    messages: [{ role: "user", content: "中国有多少个省份" }],
    model: "deepseek-chat",
  });

  console.log(completion.choices[0].message.content);
}

main();
```

:::warning 小贴士

apiKey 就是你在DeepSeek控制台创建的API KEY令牌，你可能已经把它保存起来了，复制一份到这里就行

:::

上面我们简单的调整下role为用户和提问的内容，现在使用node运行当前文件：

```
➔ test-deepseek-ai node main.js
中国共有**23个省**、**5个自治区**、**4个直辖市**和**2个特别行政区**。以下是具体列表：

### 23个省：
1. 河北省
2. 山西省
3. 辽宁省
4. 吉林省
5. 黑龙江省
6. 江苏省
7. 浙江省
8. 安徽省
9. 福建省
10. 江西省
11. 山东省
12. 河南省
13. 湖北省
14. 湖南省
15. 广东省
16. 海南省
17. 四川省
18. 贵州省
19. 云南省
20. 陕西省
21. 甘肃省
22. 青海省
23. 台湾省

### 5个自治区：
1. 内蒙古自治区
2. 广西壮族自治区
3. 西藏自治区
4. 宁夏回族自治区
5. 新疆维吾尔自治区

### 4个直辖市：
1. 北京市
2. 天津市
3. 上海市
4. 重庆市

### 2个特别行政区：
1. 香港特别行政区
2. 澳门特别行政区

总计：**34个省级行政区**。
```

成功了!!

不过上面的方式有一个明显的缺陷，就是当回复的内容很长时，会等很久后然后一次性展示出来。原因是因为接受的是文本类型字符串，若使用流就可以通过管道不断打印请求的结果

流式响应

使用流式响应也非常的简单，这里推荐读者直接查看 [OpenAI 文档](#)，因为是基于它的SDK数据接口规范来返回的，所以免去了不同模型厂商接口调用的繁琐

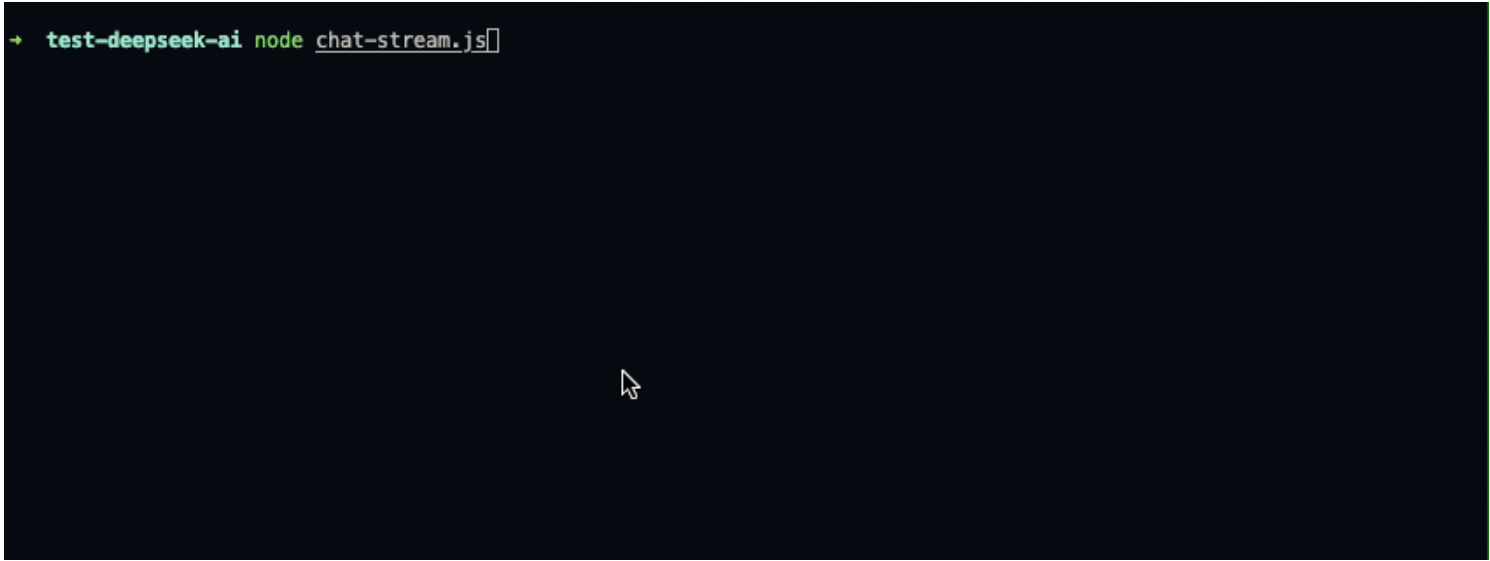
我们来进行简单的改造：

```
// 省略...
async function main() {
  const completion = await openai.chat.completions.create({
    messages: [{ role: "user", content: "中国有多少个省份" }],
    model: "deepseek-chat",
    stream: true,
  });

  // 使用流的方式写入stdout管道中
  for await (const chunk of completion) {
    process.stdout.write(chunk.choices[0]?.delta?.content || "");
  }
}

main();
```

运行后再来查看效果：



```
→ test-deepseek-ai node chat-stream.js
```

可以看出效果已经好多了!!

这样的方式在每次运行完后就会自动退出, 对于ChatGPT聊天还是不太友好, 基于此来稍微改造下, 让它变成可以不断对话的GPT

```
import readline from "readline";
import OpenAI from "openai";
import { CHAT_CONFIG } from "./config.js";

const openai = new OpenAI({
  ...CHAT_CONFIG,
});

async function generateMessage(inputText) {
  const completion = await openai.chat.completions.create({
    messages: [{ role: "user", content: inputText }],
    model: "deepseek-chat",
    stream: true,
  });

  for await (const chunk of completion) {
    process.stdout.write(chunk.choices[0]?.delta?.content || "");
  }
}
```

```
}

console.log("\n\n以上是我的回答，请合理参考，祝您生活愉快！\n\n")
}

function bootstrap() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  console.log("我是一个智能助手，你可以向我提问任何问题，我将尽力回答。🌈🌈\n\n")

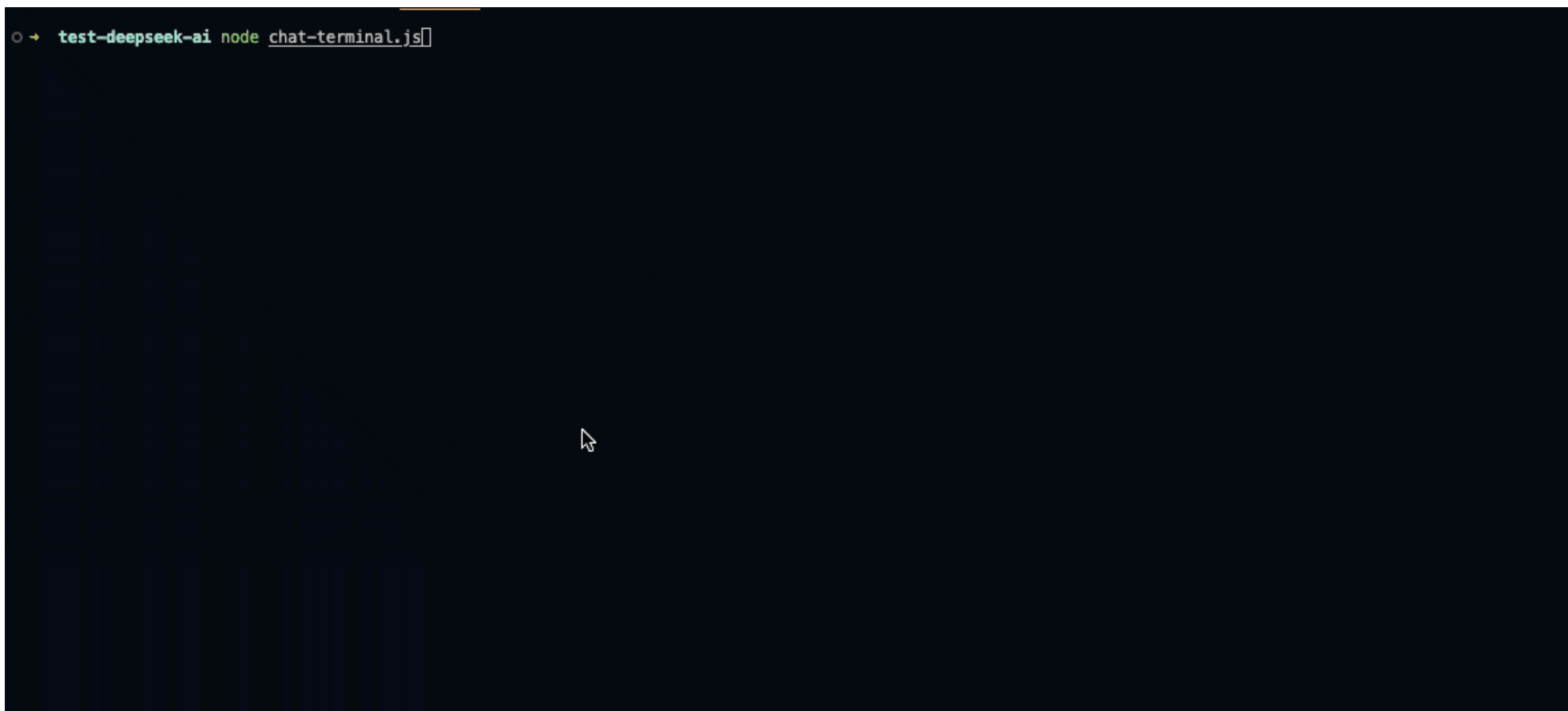
  rl.on("line", async (input) => {
    // 输入q或者Ctrl+C退出程序
    if (input === "q") {
      rl.close();
      return;
    }
    generateMessage(input);
  });

  rl.on("close", () => {
    console.log("\nBye!");
    process.exit(0);
  });
}

try {
  bootstrap();
} catch (error) {
  console.error(error);
  process.exit(1);
}
```

这里使用NodeJS提供的 `readline` 模块用来和终端进行交互，用户每输入内容回车后，程序就可以拿到用户的输入内容，然后根据内容请求大模型，这是整体思路

来看看效果：



到这里就可以完全模拟ChatGPT在终端进行交互式聊天了，而且输出的都是markdown格式内容，这对于程序员做笔记来说非常友好；相反，对于大多数普通人来说这种格式还是不太友好，并且使用的方式比较低端了

接下来我们就通过一些开源工具搭建一款高可用的AI多模型应用

ChatGPT大众化

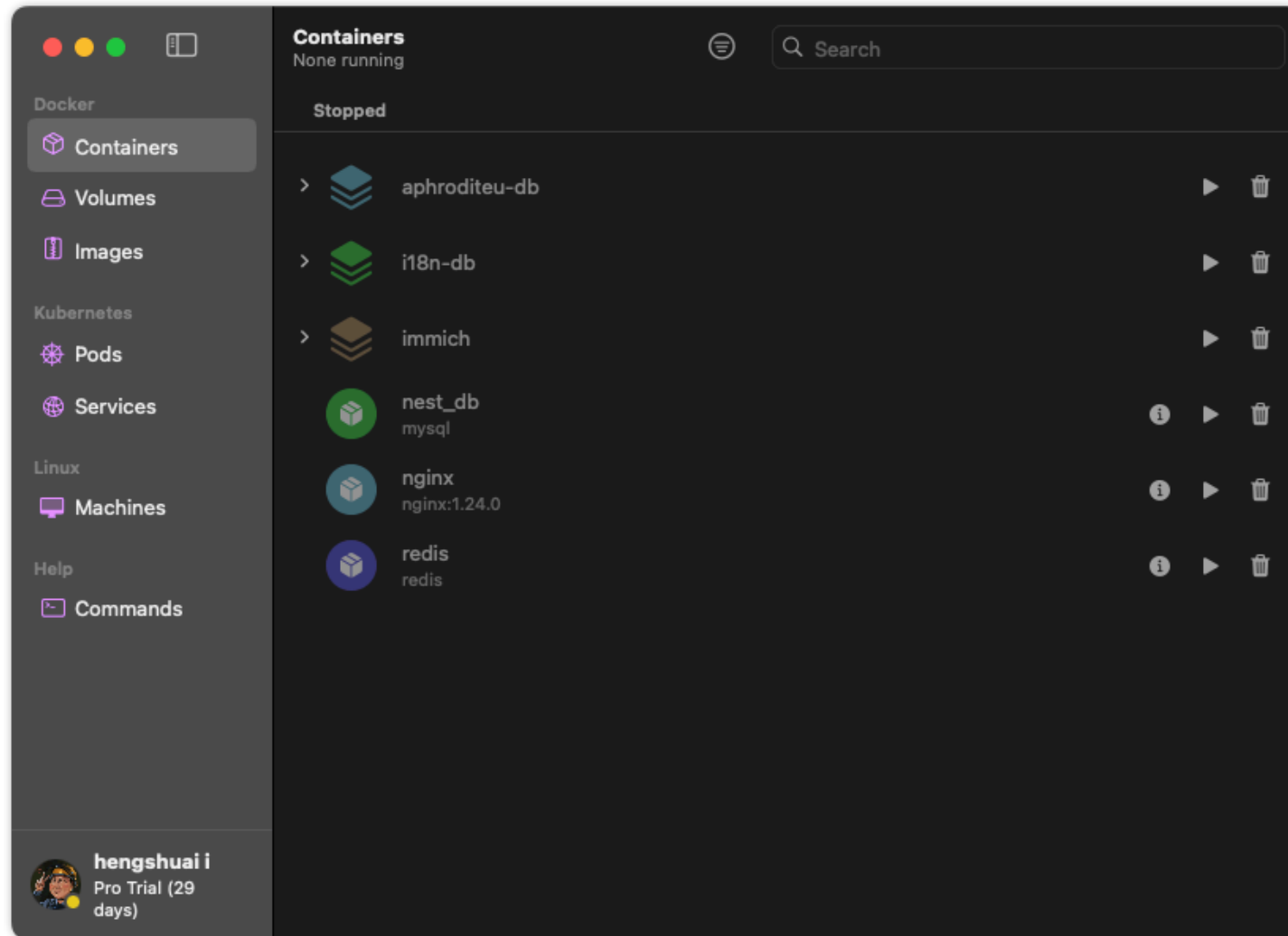
面向大众化最关键的还是更友好的客户端界面，就像ChatGPT那样以对话框的形式使用，并且还可以支持查看历史消息等等；除此之外可以通过配置直接调用我们自己付费过的大模型，是不是想着都觉得很爽

如果全部由自己手写那真的是太麻烦了，幸运的是社区有很多开源的项目我们可以使用，比如：[FastGPT](#)、[lobe-chat](#)、[MaxKB](#) 等等，感谢强大社区，感谢强大的开源🔥 本文选择 [FastGPT](#) 来搭建整体的GPT

FastGPT 是一个基于 OpenAI GPT 模型的高效、轻量级实现，旨在为用户提供快速、低成本的自然语言处理（NLP）解决方案。它通常用于生成文本、对话系统、内容创作等场景；支持多模型管理、内容生成、知识库管理、网页爬取等等。读者可以去官网查看更多它的信息

OrbStack

为了方便起见这里使用docker来安装FastGPT，读者机器上必须要有docker相关环境，若没有推荐MacOS用户使用 [OrbStack](#) 客户端，它相比官方的Docker Desktop轻量很多，并且开源免费，本人一直在使用。这里对docker的安装和配置就不多说了，可以前往阅读往期文章 [Docker安装与配置](#)



FastGPT配置与启动

首先就是下载官方的配置文件(查看[FastGPT官方部署文档](#))，直接执行以下命令：

配置文件

→ `curl -O https://raw.githubusercontent.com/labring/FastGPT/main/projects/app/data/config.json`

docker compose服务启动配置文件

→ `curl -o docker-compose.yml https://raw.githubusercontent.com/labring/FastGPT/main/files/docker/docker-compose-pgvector.yml`

:::warning 小贴士

此过程可能需要科学上网🌐📶，读者也可以通过浏览器打开后下载到本地

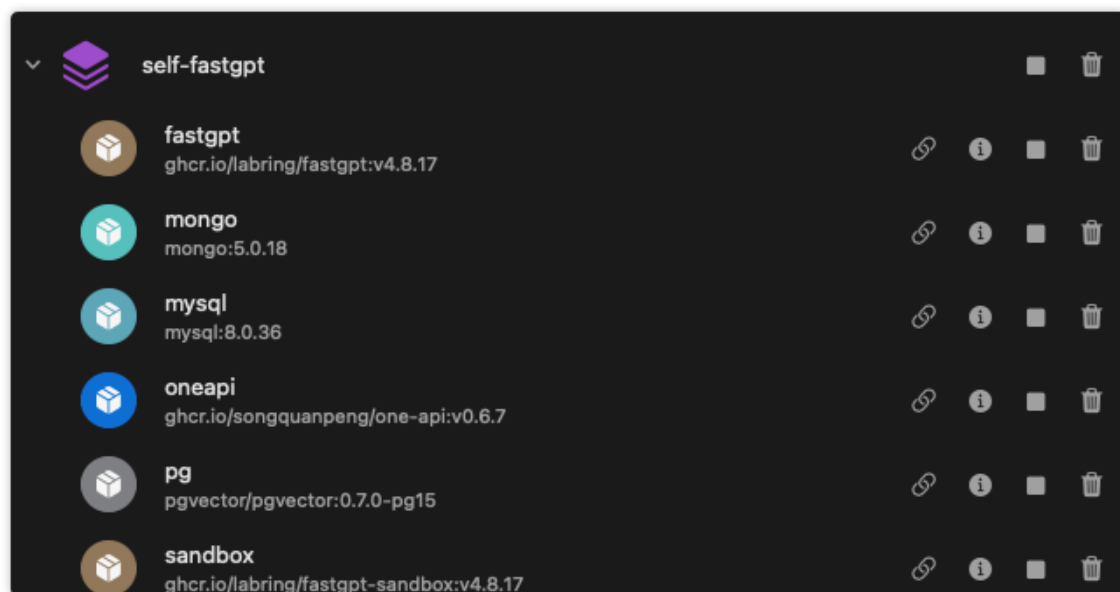
...

执行以后会生成两个文件，`config.json` (查看[config.json详细说明](#))主要用来配置使用到的模型和向量库等，`docker-compose.yml` 则是整个FastGPT运行的服务集合，里面包含前台界面、后台、网关、相关数据库等等

两个配置文件都可以进行定制化修改，比如账号密码、大模型等等，这里建议先啥都不用改，直接启动服务：

→ `docker compose up -d`

初次运行会拉取对应的镜像文件，注意网络通畅。运行成功后可以在Orbstack中看到启动的对应容器服务：



OneAPI管理模型渠道

启动成功后使用浏览器打开 `IP:3001` OneAPI，添加合适的AI模型渠道([OneAPI官方教程](#))

首次进去登录账号密码为：`root/123456`，会提示你修改账号密码

One API

🏠 首页

👥 渠道

🔑 令牌

💰 兑换

🛒 充值

👤 用户

📅 日志

⚙️ 设置

ℹ️ 关于

更新用户信息

用户名

ihengshuai

密码

.....

显示名称

ihengshuai

已绑定的 GitHub 账户

此项只读，需要用户通过个人设置页面的相关绑定按钮进行绑定，不可直接修改

已绑定的微信账户

此项只读，需要用户通过个人设置页面的相关绑定按钮进行绑定，不可直接修改

已绑定的邮箱账户

此项只读，需要用户通过个人设置页面的相关绑定按钮进行绑定，不可直接修改

取消

提交

然后点击顶部导航栏的 [渠道](#)，这里我们要添加的渠道为 [DeepSeek](#)，选择对应的类型后（你可以查看[OneAPI支持的渠道](#)），在模型那里会自动填充渠道默认的模型类别，最后输入渠道对应的令牌(API KEY)，然后保存

One API

首页

渠道

令牌

兑换

充值

用户

日志

设置

关于

root

创建新的渠道

类型

DeepSeek

名称

DeepSeekAI

分组

default

模型

deepseek-chat

deepseek-coder

填入相关模型

填入所有模型

清除所有模型

输入自定义模型名称

填入

模型重定向

此项可选，用于修改请求体中的模型名称，为一个 JSON 字符串，键为请求中模型名称，值为要替换的模型名称，例如：
{
 "gpt-3.5-turbo-0301": "gpt-3.5-turbo",
 "gpt-4-0314": "gpt-4",
 "gpt-4-32k-0314": "gpt-4-32k"
}

密钥

☐ 批量创建

代理

此项可选，用于通过代理站来进行 API 调用，请输入代理站地址，格式为：https://domain.com

保存成功后可以简单的测试下是否跑的通，点击测试按钮查看效果：

One API

首页

渠道

令牌

兑换

充值

用户

日志

设置

关于

渠道 DeepSeekAI 测试成功, 耗时 1.23 秒。

管理渠道

搜索渠道的 ID, 名称和密钥 ...

OpenAI 渠道已经不再支持通过 key 获取余额, 因此余额显示为 0。对于支持的渠道类型, 请点击余额进行刷新。
渠道测试仅支持 chat 模型, 优先使用 gpt-3.5-turbo, 如果该模型不可用则使用你所配置的模型列表中的第一个模型。

ID	名称	分组	类型	状态	响应时间	余额	优先级	操作
1	DeepSeekAI	default	DeepSeek	已启用	1.23 秒	不支持	0	测试 删除 禁用 编辑

添加新的渠道

测试所有渠道

测试禁用渠道

删除禁用渠道

刷新

<

<

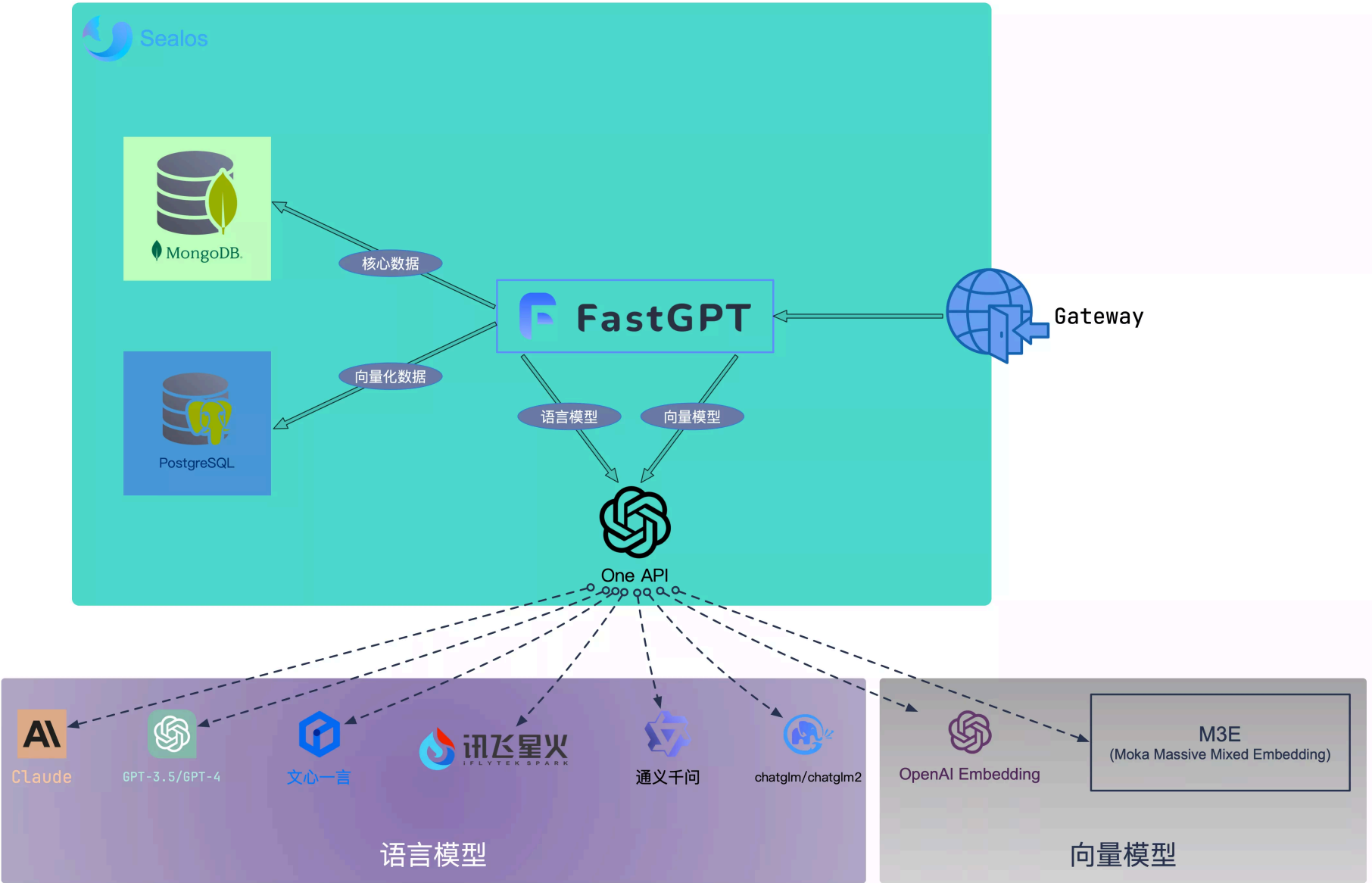
1

>

>

在FastGPT(也就是聊天界面)中请求时会带着对应的模型名称请求到OneAPI，然后请求被OneAPI转发到对应的模型接口。这里是OneAPI官方的架构解释图：

FastGPT Deployment Architecture



<https://fastgpt.run>

需要注意的是FastGPT会根据 `config.json` 配置的模型在前台显示可用的模型，默认有几个模型，读者可以直接拷贝一个配置将 `provider`、`model`、`name` 改了就行。我们来添加下DeepSeek所需的模型（也就是和渠道那里配置的模型对应）：

```
{
  "llmModels": [
    {
      "provider": "DeepSeek",
      "model": "deepseek-chat",
      "name": "deepseek-chat",
      "maxContext": 125000,
      "maxResponse": 32000,
      "quoteMaxToken": 120000,
      "maxTemperature": 1.2,
      "charsPointsPrice": 0,
      "censor": false,
      "vision": false,
      "datasetProcess": true,
      "usedInClassify": true,
      "usedInExtractFields": true,
      "usedInToolCall": true,
      "usedInQueryExtension": true,
      "toolChoice": false,
      "functionCall": false,
      "customCQPrompt": "",
      "customExtractPrompt": "",
      "defaultSystemChatPrompt": "",
      "defaultConfig": {
        "temperature": 1,
        "max_tokens": null,
        "stream": false
      }
    },
    {
      "provider": "DeepSeek",
      "model": "deepseek-coder",
      "name": "deepseek-coder",
    }
  ]
}
```

```
]
}
```

:::warning 小贴士

以上配置文件并不完整，省略了其他配置，只是增加了 `deepseek-coder` 和 `deepseek-chat` 两个模型配置，请根据自己的 `config.json` 配置文件调整

:::

调整配置后需要重启整体服务：

- `docker compose down`
- `docker compose up -d`

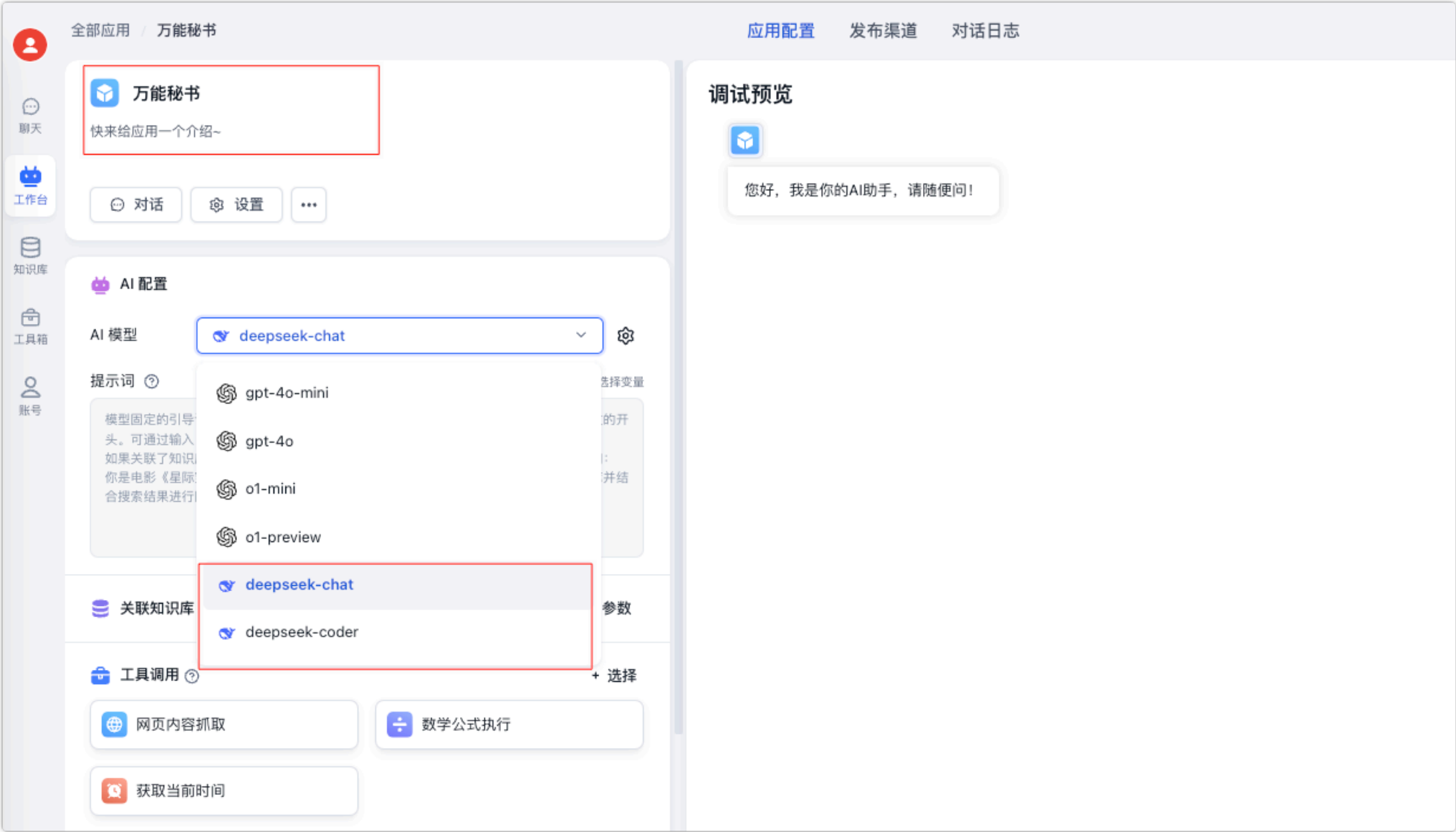
FastGPT添加模型助手

模型渠道配置管理好以后，我们再打开浏览器输入 `IP:3000` 进入FastGPT前台界面，初次进去初始账号密码为 `root/1234`

打开 `工作台` 点击右上角的 `新建/简易应用`



然后根据提示填入模型助手应用的名字、描述，模型选择我们的DeepSeek相关的模型，名字会和 `config.json` 里的配置信息是一致的（下拉选项即配置文件中的信息）；其他的如：开场白、工具调用都可以适当添加



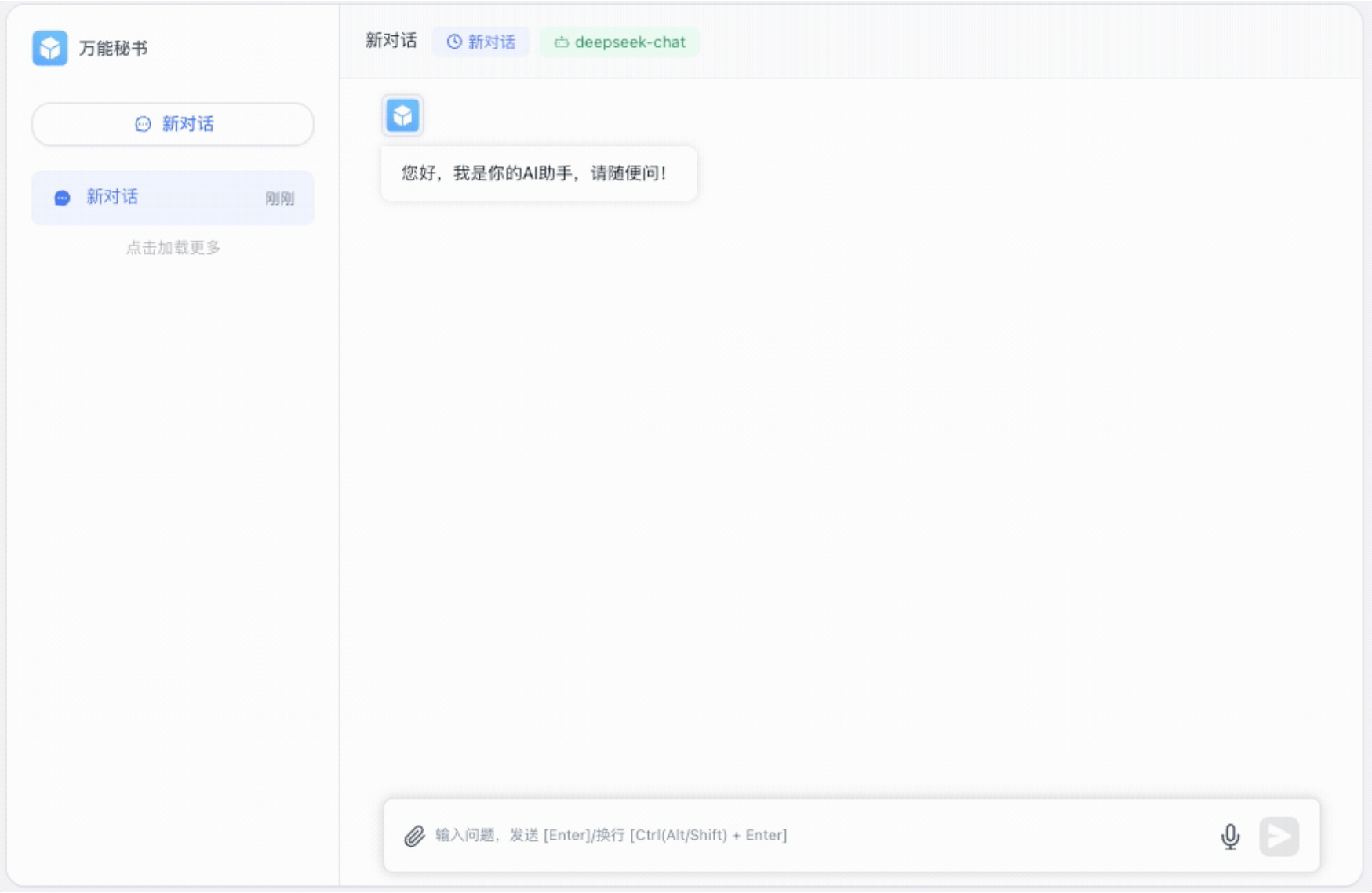
最后点击右上角的保存并发布就可以了，快去试试对话吧

开始对话

点击左侧聊天进入对话界面，就可以看到刚刚创建的模型助手了



现在我们可以提个问题，输入 请写一篇青椒炒蛋教程：



这是回答的截图效果：



不过上面回答也是以字符串的形式一次性展现，如果回复的内容过长，就会等很长时间，体验感比较差

流式响应

使用流式响应也比较简单，需要修改 `config.json` 配置文件

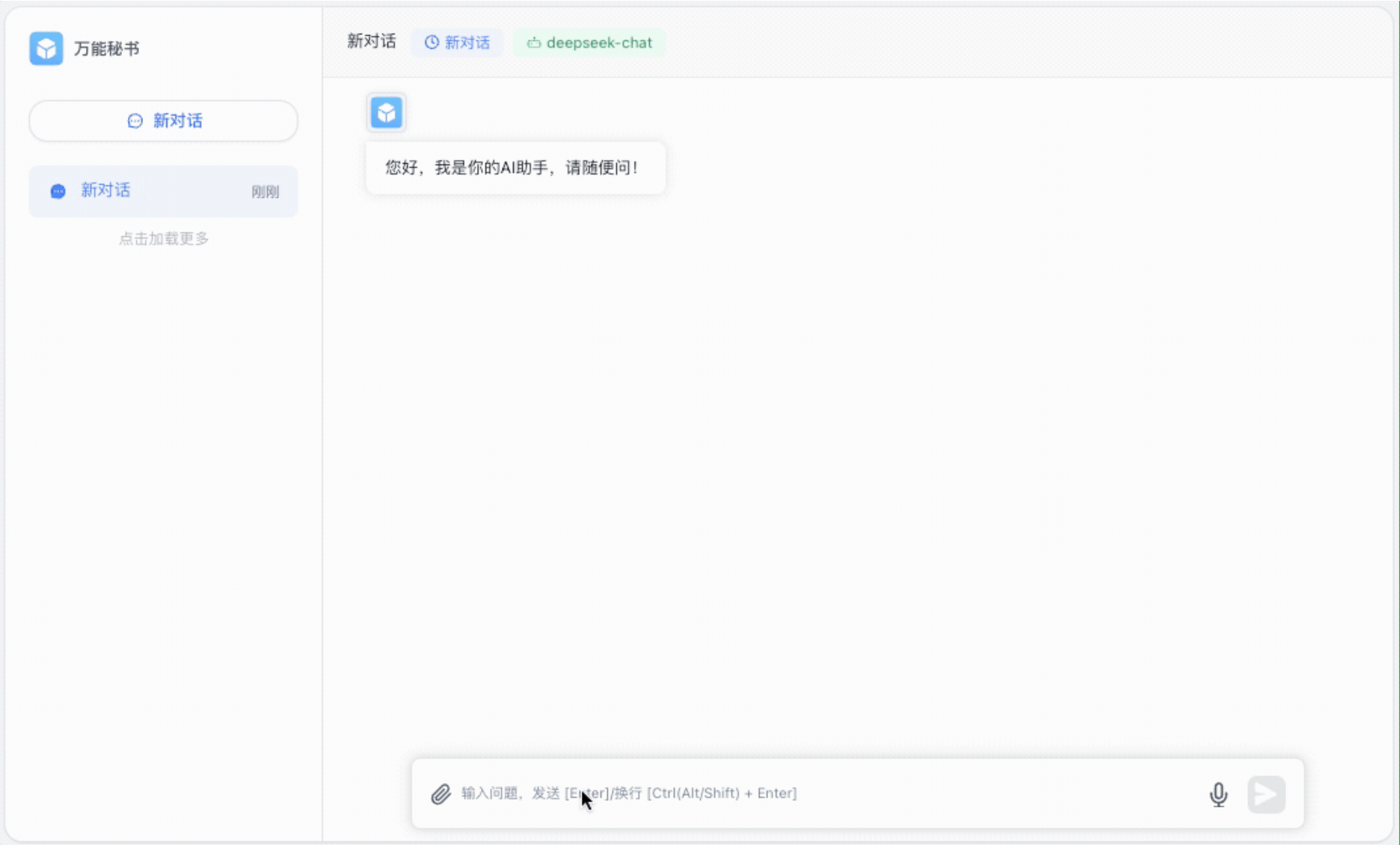
找到对应的模型配置位置，修改以下配置：

```
{
  // 部分省略
  "llmModels": [
    {
      "provider": "DeepSeek",
      "model": "deepseek-chat",
      "name": "deepseek-chat",
      // 省略
      "defaultConfig": {
        "temperature": 1,
        "max_tokens": null,
        "stream": true // 设置为true
      }
    }
  ]
}
```

修改后然后重启所有服务

→ `docker compose down`
→ `docker compose up -d`

还是上面那个例子，我们再问一次： 请写一篇青椒炒蛋教程



达到预期的效果❗

除此之外还有很多配置项可以设置，[点这里查看 config配置说明](#)

搭建知识库

FastGPT也支持知识库管理，用户可以将一些资料作为模型参考的依据，这也是私有化GPT最重要的环节！这样相较于市面上的GPT，我们就有了自己专属的知识库模型体系了。上传的知识库FastGPT会自己去训练建立索引，并且支持多种形式、格式的知识库资源，如：文本、表格、网址、web站点等等，强大到足够我们使用了

这里本人打算创建一个自己博客的知识库，这样大模型就会首先参考我的博客文章。本来是可以使用web站点这类知识库的，但他是付费功能😂

进入左侧 知识库 菜单，点击右上角 新建/通用知识库



创建进入后点击 新建/导入 ，然后选择 文本录入

根目录 / 知识库 - 博客文章

数据集 搜索测试

排队情况

文件(1)

名称

训练模式

数据总量

创建/更新时间

状态

启/停

原型、原型链与继承(构造函数继承...

直接分段

-

2025-01-07 09:31
2025-01-07 09:31

• 53 组索引中

新建/导入

文件夹

手动数据集

文本数据集

表格数据集

知识库 - 博客文章

通用知识库

这个知识库还没有介绍~

知识库 ID

677c83bfc49586d7c9b776be

索引模型

分块上限: 3000

text-embedding-3-small

文本理解模型

deepseek-coder

协作人

2

暂无协作人

之后可以上传本地文件，也可以提供网页链接；由于作者有自己的在线博客，这里我选择网页链接

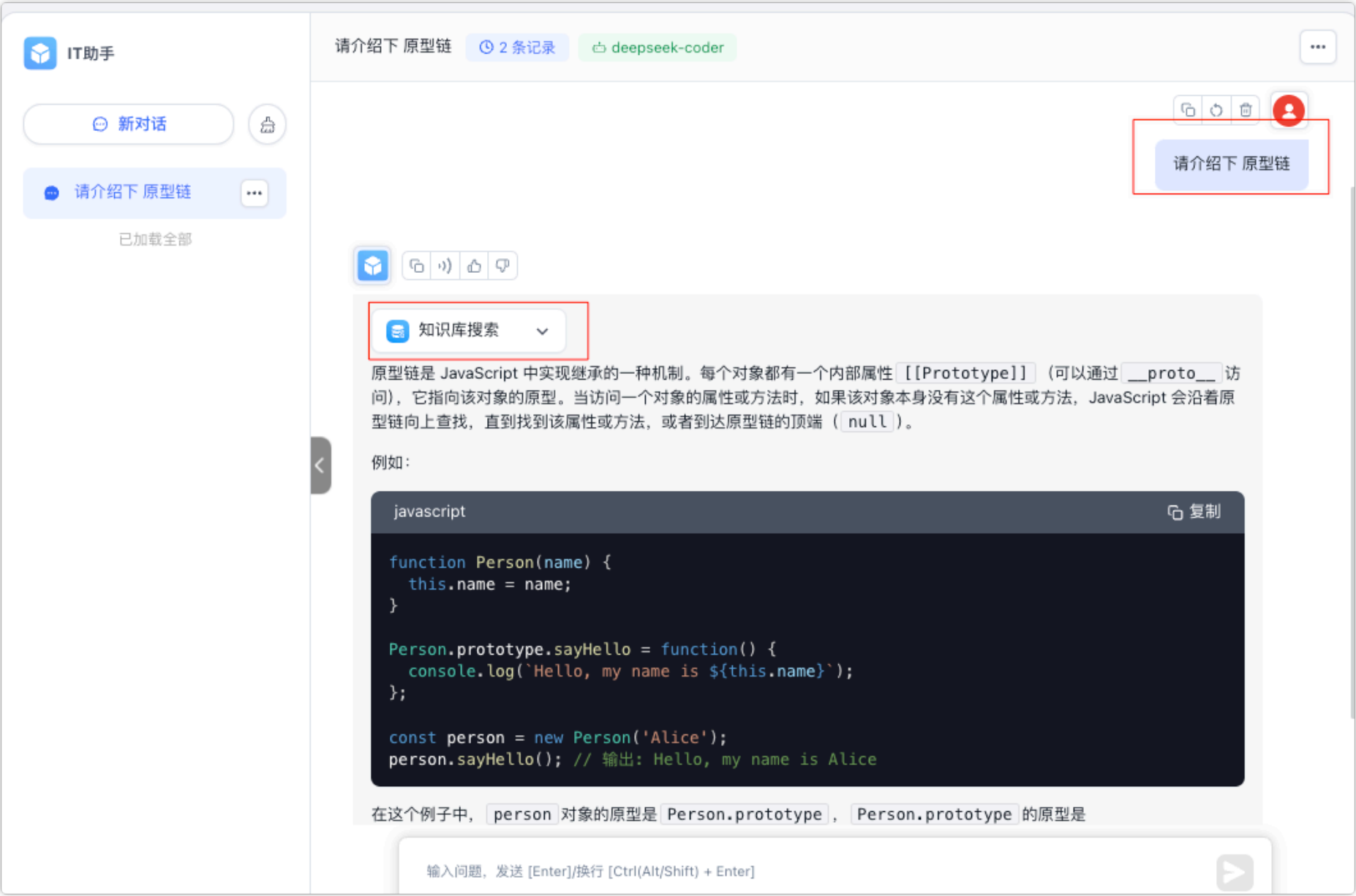


然后输入指定文章的链接，这里我输入的是「原型、原型链与继承 <https://blog.usword.cn/frontend/js/proto-inherit.html>」，点击保存后就可以看到刚刚添加的知识库了

有了知识库后需要将知识库关联到某个模型助手中，这里我们重新创建一个 **IT助手** 模型应用，然后关联下刚刚创建的知识库 **博客知识库**



现在打开 IT助手 对话框，这里我输入 请介绍下 原型链 ，然后助手就参考了下知识库的内容，并生成最后的结果



其他

FastGPT还支持很多功能, 如: 向量化数据库等等, 这里就不展开, 可以查看官方文档

到这里一款私有的ChatGPT就搭建成功了，并且可以根据知识库来定制自己的模型结果，真是太香了

总结

在人工智能技术飞速发展的今天，搭建私有 ChatGPT 已成为许多企业和组织的优先选择。私有 ChatGPT 不仅能够提供与 OpenAI 的 ChatGPT 相媲美的强大文本生成和对话能力，还具备数据隐私保护、定制化开发、高性能运行等独特优势。通过私有化部署，企业可以完全掌控自己的数据和模型，避免敏感信息泄露，同时根据自身需求定制专属的 AI 助手，提升业务效率和用户体验

无论是通过微调现有模型，还是基于开源框架（如 FastGPT、LLaMA）进行二次开发，都可以快速构建属于自己的智能对话系统。总之，搭建私有 ChatGPT 不仅能满足个性化需求，还能为数据安全和业务发展提供坚实保障

由于格式、图片和在线案例解析问题，问了更好的阅读体验，读者可前往 [阅读原文](#)

=====

最新的deepseek本地大模型部署，可查看 [《本地部署DeepSeek摆脱服务器繁忙》](#)一文